2. Add Two Numbers

PLATFORM	DIFFICULTY	DATE
Platform: LeetCode	MEDIUM	Solved: July 8, 2026

Problem Summary

Given two non-empty linked lists representing two non-negative integers stored in **reverse order**, add them and return the sum as a linked list in reverse order.

Approach

Simulate grade-school addition digit by digit — traverse both lists simultaneously, add corresponding digits plus any carry, create a new node for each digit of the result, and propagate carry forward. Use a dummy head node to simplify result list construction.

Algorithm

Initialize dummy = new ListNode(0), current = dummy, carry = 0.
While l1 != null || l2 != null || carry != 0:

- val1 = l1 != null ? l1.val : 0
- val2 = l2 != null ? l2.val : 0
- total = val1 + val2 + carry
- carry = total / 10, newDigit = total % 10
- Append new ListNode(newDigit) to current, advance current.
- Advance l1, l2 if not null.

Return dummy.next.

Key Insights

- Lists are already in reverse order — LSB is at head — so you can add left to right naturally without reversing.
- carry != 0 as a third loop condition handles the case where a final carry creates an extra digit (e.g., 99 + 1 = 100).
- Dummy head node eliminates special-casing the first node — dummy.next is always the real head.
- Ternary (l1 != null) ? l1.val : 0 pads the shorter list with 0 cleanly.

Points to Remember

1. **Dummy head pattern**: use ListNode dummy = new ListNode(0) whenever building a new linked list — return dummy.nextat the end. Avoids null checks for the first node.
2. **Three-condition while loop**: l1 != null || l2 != null || carry != 0 — the carry != 0 part is easy to forget and causes wrong answers on inputs like [9,9] + [1].
3. **Digit extraction**: total / 10 → carry, total % 10 → digit — reusable in any digit-by-digit arithmetic problem.
4. Pad shorter list with 0 using ternary — cleaner than writing separate loops for leftover nodes.

Observations

- Loop runs as long as **any** of the three conditions is true — handles unequal lengths and leftover carry cleanly in one condition.
- total / 10 extracts carry, total % 10 extracts digit — standard digit extraction pattern.
- current always points to the last added node — classic result-list building pattern.
- No need to reverse anything since input is already reversed.

Time Complexity & Space Complexity

O(max(m, n)) — where m, n are lengths of l1 and l2; loop runs for the longer list + at most one extra step for carry.

O(max(m, n)) — result list has at most max(m, n) + 1 nodes.

Linked List

Description | Editorial | Solutions | Submissions

2. Add Two Numbers

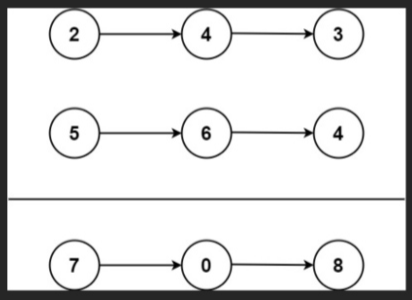
Solved

Medium | Topics | Companies

You are given two **non-empty** linked lists representing two non-negative integers. The digits are stored in **reverse order**, and each of their nodes contains a single digit. Add the two numbers and return the sum as a linked list.

You may assume the two numbers do not contain any leading zero, except the number 0 itself.

Example 1:




```
Input: l1 = [2,4,3], l2 = [5,6,4]
Output: [7,0,8]
Explanation: 342 + 465 = 807.
```

Example 2:

```
Input: l1 = [0], l2 = [0]
Output: [0]
```

37K | 1.1K | 841 Online

Submit | Premium

Accepted

All Submissions

Accepted 1569 / 1569 testcases passed

submitted at Jul 09, 2026

Analysis | Solution

Runtime

1 ms | Beats 100.00%

Memory

46.70 MB | Beats 19.77%

Code | Java

```java
/**
 * Definition for singly-linked list.
 * public class ListNode {
 *     int val;
 *     ListNode next;
 *     ListNode() {}
 *     ListNode(int val) { this.val = val;
 *     ListNode(int val, ListNode next) { t
```

View more

Write your notes here

Code

Java | Auto

```java
/**
 * Definition for singly-linked list.
 * public class ListNode {
 *     int val;
 *     ListNode next;
 *     ListNode() {}
 *     ListNode(int val) { this.val = val; }
 *     ListNode(int val, ListNode next) { this.val = val; this.next = next; }
 * }
 */
class Solution {
    public ListNode addTwoNumbers(ListNode l1, ListNode l2) {
        ListNode dummy = new ListNode(0);;
        ListNode current = dummy;
        int carry = 0;

        while(l1!=null || l2!=null || carry!=0){
            int val1 = (l1!=null)?l1.val:0;
            int val2 = (l2!=null)?l2.val:0;

            int total = val1+val2+carry;
            carry = total/10;
            int newDigit = total%10;

            current.next = new ListNode(newDigit);
            current = current.next;

            if(l1!=null) l1=l1.next;
            if(l2!=null) l2=l2.next;
        }

        return dummy.next;
    }
```

Saved

Ln 1, Col 1

Testcase | Test Result

You must run your code first